CoreInventory

Hackathon Task Breakdown — Frontend | Backend | AI/ML

Based on Excalidraw Mockup • 8-Hour Hackathon Plan • 2026

1. Mockup Se Samjhe — Kya Kya Screens Hain?

Excalidraw mockup me ye screens clearly defined hain:

#	Screen Name	Key Elements (Mockup Se)	Team Responsible
1	Login Page	Login ID field, Password field, SIGN IN btn, Forget Password, Sign Up link	Frontend + Backend
2	Sign Up Page	Enter Login ID, Email, Password, Re-Enter Password, SIGN UP btn. Creates user in DB	Frontend + Backend
3	Dashboard	Nav: Dashboard/Operations/Products/Move History/Settings. KPI cards: Total Stock, Low Stock, Pending Receipts/Deliveries, Transfers	Frontend + Backend + AI
4	Receipts (List)	NEW btn, List view default, Reference (WH/IN/0001), From (vendor), Status (Ready/Draft/Done), Kanban view toggle, search by reference & contact	Frontend + Backend
5	Receipt (Detail/Form)	WH/IN/0001 header, Receive From, Responsible (auto-fill logged user), Schedule Date, Product lines + Qty, New/Validate/Print/Cancel btns. Status: Draft > Ready > Done	Frontend + Backend
6	Delivery Orders (List)	NEW btn, List view, Reference (WH/OUT/0001), From (WH/Stock1), To (vendor), Contact, Schedule date, Status. Kanban view toggle	Frontend + Backend
7	Delivery (Detail/Form)	WH/OUT/0001 header, Delivery Address, Responsible, Schedule Date, Operation type, Product lines (alert if out of stock). Status: Draft > Waiting > Ready > Done	Frontend + Backend + AI
8	Move History	List of all stock movements — Reference, Date, Contact, From, To, Quantity, Status. Filter & search. Kanban view	Frontend + Backend
9	Products Page	Product list table: Product name, Per Unit Cost, On Hand qty, Free to Use qty. NEW btn, Update stock from here	Frontend + Backend
10	Product Form	Product Name, SKU [DESK001], Quantity (e.g. 6), Category, Unit. Alert if not in stock on delivery	Frontend + Backend + AI
11	Settings — Warehouse	Warehouse: Name, Short Code, Address. Page note: Contains warehouse details & location	Frontend + Backend

12	Settings — Location	Location: Name, Short Code, Warehouse (parent). Holds multiple locations — rooms, racks etc.	Frontend + Backend

2. Reference Number Format (Mockup Se)

Mockup me clearly shown hai ki auto-increment references kaise honge:

Operation	Format	Example
Receipt (Incoming)	**WH/IN/0001**	Warehouse/Incoming/SequenceNumber
Delivery (Outgoing)	**WH/OUT/0001**	Warehouse/Outgoing/SequenceNumber
Internal Transfer	**WH/INT/0001**	Warehouse/Internal/SequenceNumber
Adjustment	**WH/ADJ/0001**	Warehouse/Adjustment/SequenceNumber

3. 8-Hour Hackathon Timeline — Kaun Kab Kya Karega?

Ye parallel execution plan hai. Teeno teams simultaneously kaam karein:

Hour	Frontend	Backend	AI/ML	Milestone
0–1h	Project setup, routing, base layout, sidebar nav	Project setup, DB schema design, auth endpoints	Environment setup, model/library selection	All repos running locally
1–2h	Login/Signup UI + OTP screen	Auth API: login, signup, OTP, JWT	Low stock detection logic design	Auth working end-to-end
2–3h	Dashboard page + KPI cards UI	Dashboard KPI APIs (stock count, pending counts)	Low stock threshold engine (rule-based)	Dashboard showing live data
3–4h	Products page + Product form UI	Products CRUD API + SKU auto-gen	SKU search/smart filter backend logic	Products create & list working
4–5h	Receipts list + detail/form screens	Receipts API: create, validate, stock auto +	Stock movement anomaly detection (basic)	Receipts flow end-to-end
5–6h	Delivery Orders list + detail screens	Delivery API: create, validate, stock auto -	Alert trigger on out-of-stock delivery	Delivery flow end-to-end
6–7h	Internal Transfers + Move History screens	Transfer API + Stock Ledger logging	Move history analytics / summary	Transfers + ledger working
7–8h	Polish UI, low-stock alert banners, filters, kanban view	Stock Adjustments API + Settings (Warehouse/Location)	AI alert notifications integration	Full app demo-ready

4. FRONTEND Tasks — Detailed

Tech Stack: React.js / Next.js + Tailwind CSS + Axios

FE-01 Project Setup + Base Layout + Sidebar Navigation [Frontend]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	**45 min**	All Screens — Left Sidebar nav visible on every screen

What to build:

- Create React/Next project with Tailwind CSS setup
- Sidebar layout: Dashboard, Operations (dropdown: Receipts, Delivery, Adjustment), Products, Move History, Settings
- Top navbar with user avatar + logout
- React Router / Next routing setup for all pages
- Axios instance with base URL + JWT interceptor (auto attach token)
- Global state: React Context or Zustand for user, warehouse info

Checklist / Deliverables:

- ☐ Sidebar renders on all authenticated pages
- ☐ Route protection (redirect to login if no token)
- ☐ Navigation items highlighted on active route
- ☐ Responsive layout base done

FE-02 Login + Signup + OTP Screens [Frontend]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	45 min	Login Page + Sign Up Page (Mockup: top-left screens)

What to build:

- Login: Login ID field + Password field + SIGN IN button
- Forgot Password link + Sign Up link on login page
- Sign Up: Login ID, Email ID, Password, Re-Enter Password, SIGN UP button
- OTP screen: 6-digit OTP input, resend link
- Form validation (empty fields, password match check)
- API call to /auth/login, /auth/signup, /auth/verify-otp
- Store JWT in localStorage, redirect to Dashboard on success
- Show error messages from API on screen

Checklist / Deliverables:

- ☐ Login form submits and receives JWT token
- ☐ Signup creates account + shows success
- ☐ OTP flow works for password reset
- ☐ Redirect logic correct

FE-03 Dashboard Page — KPI Cards + Filters [Frontend]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	60 min	Dashboard Screen — KPI cards, dynamic filters (Mockup: main area)

What to build:

- 5 KPI cards: Total Products in Stock, Low Stock / Out of Stock, Pending Receipts, Pending Deliveries, Transfers Scheduled
- Each card shows count + colored status (red for low/out, blue for pending, etc.)
- Filter bar: By document type (Receipts/Delivery/Internal/Adjustments), By status (Draft/Waiting/Ready/Done/Canceled), By warehouse, By category
- Recent operations list below KPI cards
- Low stock alert banner (AI-triggered — highlight in red)
- API: GET /dashboard/stats, GET /dashboard/recent

Checklist / Deliverables:

- ☐ All 5 KPI cards showing real data from API
- ☐ Filters change the recent operations list
- ☐ Low stock alerts highlighted in red
- ☐ Auto-refresh every 30 seconds

FE-04 Products Page + Product Form [Frontend]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	60 min	Products Screen — table with Per Unit Cost, On Hand, Free to Use columns

What to build:

- Products list table: columns — Product Name, SKU/Code, Category, Per Unit Cost, On Hand Qty, Free to Use Qty
- NEW button opens product creation form/modal
- Product form fields: Name, SKU (auto-generate or manual), Category (dropdown), Unit of Measure, Initial Stock, Per Unit Cost
- Edit product inline or via modal

- Stock availability update directly from products page (as per mockup note)
- Search bar — search by name or SKU
- Alert badge if product is out of stock
- API: GET /products, POST /products, PUT /products/:id

Checklist / Deliverables:

- ☐ Product list renders with all columns
- ☐ Create product form saves to backend
- ☐ SKU shows as [DESK001] format like mockup
- ☐ On hand vs free-to-use counts showing

FE-05 Receipts — List View + Detail/Form [Frontend]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	75 min	Receipts Screen (Mockup: list with WH/IN/0001, vendor, status) + Detail form

What to build:

- List view: columns — Reference (WH/IN/0001), From (vendor name), Status badge (Draft/Ready/Done), Schedule Date, # of products
- NEW button creates a new receipt (draft state)
- List view default; toggle to Kanban view by status
- Search/filter by reference number and contact/vendor name
- Detail/Form screen: Reference (auto-generated WH/IN/00XX), Receive From dropdown (vendor), Responsible (auto-fill = logged-in user), Schedule Date picker
- Product lines table: add product, input quantity received
- Action buttons: New, Validate (move to Done, stock +), Print (show print-friendly view), Cancel
- Status flow indicator: Draft → Ready → Done (shown at top)
- On Validate: POST /receipts/:id/validate — stock auto-increases

Checklist / Deliverables:

- ☐ Receipt list shows all receipts with status
- ☐ New receipt creates draft
- ☐ Responsible auto-fills with logged-in user name
- ☐ Validate button increases stock
- ☐ Print-friendly view renders
- ☐ Kanban view groups by status

FE-06 Delivery Orders — List View + Detail/Form [Frontend]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	75 min	Delivery Screen (Mockup: WH/OUT/0001, WH/Stock1, vendor, Ready status)

What to build:

- List view: columns — Reference (WH/OUT/0001), From (WH/Stock1), To (vendor/customer), Contact, Schedule Date, Status
- NEW button creates new delivery (draft state)
- Toggle List/Kanban view by status
- Search/filter by reference and contact
- Detail screen: Reference, Delivery Address, Responsible (auto-fill), Schedule Date, Operation type
- Product lines: add product + quantity. If product out of stock — highlight line RED with alert icon
- Status flow: Draft → Waiting → Ready → Done
- Waiting state: product is out of stock and waiting to be restocked
- Action buttons: New, Validate (stock -), Print, Cancel
- On Validate: stock decreases automatically

Checklist / Deliverables:

- ☐ Delivery list renders with WH/OUT reference format
- ☐ Out of stock products highlighted red
- ☐ Validate decreases stock correctly
- ☐ Status flow Draft > Waiting > Ready > Done works
- ☐ Kanban view by status works

FE-07 Move History Screen [Frontend]

Priority	Time Estimate	Screen / Module (from Mockup)
P2-HIGH	45 min	Move History (Mockup: list of all in/out movements with From, To, Reference, Date)

What to build:

- Table: Reference, Date, Contact/Vendor, From Location, To Location, Quantity, Status
- If one reference has multiple products — show grouped rows
- Default list view, toggle to Kanban
- Filter by reference, contact, date range, status
- Read-only view — no edit, just history
- API: GET /stock-ledger?filters=...
- Populate moves done between From-To locations in inventory

Checklist / Deliverables:

- ☐ All stock movements visible in chronological order
- ☐ Grouped by reference if multiple products
- ☐ Date range filter works
- ☐ Kanban view by status works

FE-08 Settings — Warehouse + Location [Frontend]

Priority	Time Estimate	Screen / Module (from Mockup)
P2-HIGH	45 min	Settings Screen (Mockup: Warehouse: Name, Short Code, Address + Location sub-page)

What to build:

- Settings page with two sub-sections: Warehouse and Location
- Warehouse form: Name, Short Code, Address — create/edit/list
- Location form: Name, Short Code, Warehouse (parent dropdown) — create/edit/list
- Note from mockup: Locations holds multiple rooms, racks etc. inside a warehouse
- List all warehouses and their locations
- API: GET/POST/PUT /warehouses, GET/POST/PUT /locations

Checklist / Deliverables:

- ☐ Warehouse create works
- ☐ Location linked to correct warehouse
- ☐ Both show in dropdowns on Receipts/Delivery forms

FE-09 Stock Adjustments Screen [Frontend]

Priority	Time Estimate	Screen / Module (from Mockup)
P2-HIGH	40 min	Adjustment — fix stock mismatch (not explicitly drawn but mentioned in Operations submenu)

What to build:

- Form: Select Product, Select Location, Enter Counted Quantity
- System shows current recorded quantity vs entered quantity
- Difference auto-calculated and shown
- Confirm Adjustment button: POST /adjustments
- Adjustment logged in Move History with reference WH/ADJ/XXXX

Checklist / Deliverables:

- ☐ Adjustment form works end-to-end
- ☐ Stock updates after adjustment
- ☐ Appears in Move History as ADJ reference

5. BACKEND Tasks — Detailed

Tech Stack: Node.js + Express.js (or FastAPI/Django) + PostgreSQL / MongoDB + JWT Auth

BE-01 Project Setup + DB Schema + Auth API [Backend]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	60 min	Auth — Login Page, Sign Up Page

What to build:

- Initialize Express/FastAPI project with folder structure: routes/, controllers/, models/, middleware/
- Database connection setup (PostgreSQL or MongoDB)
- Create all DB tables/collections upfront (users, products, warehouses, locations, receipts, deliveries, transfers, adjustments, stock_ledger)
- POST /auth/signup: hash password (bcrypt), save user, return JWT
- POST /auth/login: verify credentials, return JWT
- POST /auth/forgot-password: generate OTP, store with expiry (10 min)
- POST /auth/verify-otp: validate OTP, allow password reset
- POST /auth/reset-password: update hashed password
- JWT middleware: protect all routes except auth

Checklist / Deliverables:

- ☐ All 5 auth endpoints working and tested with Postman
- ☐ Passwords hashed (never plain text in DB)
- ☐ JWT expires in 24h
- ☐ OTP expires in 10 minutes

Database Schema (All Tables)

users — Field	Type	Constraint	Description
id	UUID / Int	PRIMARY KEY	Auto-generated
login_id	VARCHAR(50)	UNIQUE, NOT NULL	Unique login identifier
email	VARCHAR(100)	UNIQUE, NOT NULL	User email for OTP
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password
name	VARCHAR(100)		Display name
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time

warehouses — Field	Type	Constraint	Description
id	UUID	PRIMARY KEY	
name	VARCHAR(100)	NOT NULL	e.g. Main Warehouse
short_code	VARCHAR(10)	UNIQUE	e.g. WH
address	TEXT		Physical address

locations — Field	Type	Constraint	Description
id	UUID	PRIMARY KEY	
name	VARCHAR(100)	NOT NULL	e.g. Rack A, Production Floor
short_code	VARCHAR(10)		e.g. RACK-A

warehouse_id	UUID	FK → warehouses.id	Parent warehouse

products — Field	Type	Constraint	Description
id	UUID	PRIMARY KEY	
name	VARCHAR(200)	NOT NULL	e.g. Desk
sku	VARCHAR(50)	UNIQUE, NOT NULL	e.g. DESK001
category	VARCHAR(100)		Product category
unit_of_measure	VARCHAR(30)		kg / pcs / box
per_unit_cost	DECIMAL(10,2)		Cost per unit
reorder_level	INT	DEFAULT 10	Low stock threshold for AI alert

stock_on_hand — Field	Type	Constraint	Description
id	UUID	PRIMARY KEY	
product_id	UUID	FK → products.id	Which product
location_id	UUID	FK → locations.id	Which location
quantity	DECIMAL(10,2)	DEFAULT 0	Current stock at this location

stock_ledger (Move History) — Field	Type	Constraint	Description
id	UUID	PRIMARY KEY	
reference	VARCHAR(20)	NOT NULL	WH/IN/0001 or WH/OUT/0001
operation_type	ENUM	receipt/delivery/transfer/adjustment	Type of movement
product_id	UUID	FK → products.id	Product moved
from_location_id	UUID	FK → locations.id	Source location
to_location_id	UUID	FK → locations.id	Destination location
quantity	DECIMAL(10,2)	NOT NULL	Qty moved
status	ENUM	draft/waiting/ready/done/canceled	Current status
responsible_user_id	UUID	FK → users.id	Who did it
contact	VARCHAR(200)		Vendor/Customer name
schedule_date	DATE		Scheduled date
created_at	TIMESTAMP	DEFAULT NOW()	When logged

BE-02 Products CRUD API [Backend]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	**45 min**	Products Screen — create, read, update, stock view

What to build:

- GET /products — list all products with on-hand qty + free-to-use qty per location
- POST /products — create product (name, sku auto-gen if empty, category, unit, cost, initial stock)
- PUT /products/:id — update product details
- GET /products/:id/stock — stock breakdown per location
- Auto-generate SKU: take first 4 chars of name + 3-digit sequence (e.g. DESK001)
- Free to use = on_hand - reserved_for_delivery (reserved qty from pending delivery orders)
- GET /products/low-stock — products below reorder_level (for AI module)

Checklist / Deliverables:

- ☐ GET /products returns correct on_hand and free_to_use
- ☐ SKU auto-generation works
- ☐ Create + Update tested

BE-03 Receipts API — Incoming Stock [Backend]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	60 min	Receipts Screen — create, list, validate

What to build:

- GET /receipts — list all receipts with status, from, reference, product count
- POST /receipts — create draft receipt (auto-generate reference WH/IN/XXXX, set status=draft, responsible=logged user)
- PUT /receipts/:id — update lines (add/remove products, set quantities, set vendor, schedule date)
- POST /receipts/:id/validate — validate receipt: for each product line, increase stock_on_hand at destination location, write to stock_ledger, set status=done
- POST /receipts/:id/cancel — set status=canceled
- GET /receipts/:id — get single receipt detail with all product lines
- Reference format: query MAX reference, increment, pad to 4 digits
- Transition rules: draft → ready (on TODO action) → done (on validate)

Checklist / Deliverables:

- ☐ POST /receipts creates draft with correct WH/IN/XXXX reference
- ☐ Validate increases stock correctly in stock_on_hand
- ☐ stock_ledger row created on validate
- ☐ Status transitions work

BE-04 Delivery Orders API — Outgoing Stock [Backend]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	60 min	Delivery Screen — create, list, validate with stock check

What to build:

- GET /deliveries — list all with reference, from location, to, contact, status
- POST /deliveries — create draft (auto-generate WH/OUT/XXXX, status=draft, responsible=logged user)
- PUT /deliveries/:id — update lines (add products, quantities, delivery address, schedule date, operation type)
- Stock check on adding product lines: if free_to_use < requested qty → set line status=waiting, flag product as out_of_stock
- POST /deliveries/:id/validate — decrease stock_on_hand, write to stock_ledger, status=done. Fail gracefully if stock insufficient
- POST /deliveries/:id/cancel — status=canceled
- Status flow: draft → waiting (if stock issue) → ready → done
- Trigger AI alert if product goes below reorder_level after delivery

Checklist / Deliverables:

- ☐ Delivery creation with WH/OUT reference
- ☐ Out-of-stock detection flags correctly
- ☐ Validate decreases stock
- ☐ stock_ledger entry created

BE-05 Internal Transfers + Stock Adjustments + Move History API [Backend]

Priority	Time Estimate	Screen / Module (from Mockup)

P2-HIGH	**60 min**	Move History Screen + Adjustment feature

What to build:

- POST /transfers — create internal transfer (from_location, to_location, product, qty, WH/INT reference)
- POST /transfers/:id/validate — move qty from source to destination in stock_on_hand. Total qty unchanged. Log in stock_ledger
- POST /adjustments — stock adjustment: select product + location, enter physical count, system calculates diff, apply diff to stock_on_hand, log in stock_ledger with WH/ADJ reference
- GET /stock-ledger — list all movements: reference, date, type, product, from, to, qty, status. Support filters: date range, type, reference, contact, status
- If single reference has multiple products: group them but return as separate lines
- GET /dashboard/stats — return: total_products_in_stock, low_stock_count, out_of_stock_count, pending_receipts_count, pending_deliveries_count, scheduled_transfers_count
- GET /dashboard/recent — recent 10 operations across all types

Checklist / Deliverables:

- ☐ Transfer moves stock between locations (total same)
- ☐ Adjustment updates stock and logs correctly
- ☐ Move History shows all operations filtered/grouped
- ☐ Dashboard stats endpoint returns correct counts

BE-06 Warehouses + Locations Settings API [Backend]

Priority	Time Estimate	Screen / Module (from Mockup)
P2-HIGH	**30 min**	Settings Screen — Warehouse, Location CRUD

What to build:

- GET /warehouses, POST /warehouses, PUT /warehouses/:id
- GET /locations?warehouse_id=XX, POST /locations, PUT /locations/:id
- Locations must belong to a warehouse (FK constraint)
- Return warehouse + its locations nested in GET /warehouses/:id
- Used in Receipts (destination), Delivery (from), Transfer (from/to) dropdowns

Checklist / Deliverables:

- ☐ Warehouse CRUD works
- ☐ Location linked to warehouse
- ☐ Locations appear in receipt/delivery dropdowns on FE

6. API Contracts — FE-BE Interface

Frontend aur Backend in endpoints pe agree karein PEHLE coding shuru karne se. Base URL: /api/v1

Auth Endpoints

Endpoint	Method	Request Body (key fields)	Response / Notes
/auth/signup	POST	login_id, email, password, name	{ token, user: {id, name, email} }
/auth/login	POST	login_id, password	{ token, user: {id, name, email} }
/auth/forgot-password	POST	email	{ message: 'OTP sent' }
/auth/verify-otp	POST	email, otp	{ reset_token }
/auth/reset-password	POST	reset_token, new_password	{ message: 'Success' }

Products Endpoints

Endpoint	Method	Request Body (key fields)	Response / Notes
/products	GET	—	[{id, name, sku, category, unit, cost, on_hand, free_to_use}]
/products	POST	name, category, unit, cost, initial_stock, reorder_level	{ product }
/products/:id	PUT	any updatable field	{ product }
/products/low-stock	GET	—	[products where on_hand < reorder_level]

Receipts Endpoints

Endpoint	Method	Request Body (key fields)	Response / Notes
/receipts	GET	status?, from?, date?	[{id, reference, from, status, schedule_date, line_count}]
/receipts	POST	vendor, schedule_date, lines:[{product_id, qty}]	{ receipt with WH/IN/XXXX reference, status: draft }
/receipts/:id	PUT	vendor, lines, schedule_date	{ receipt }
/receipts/:id/validate	POST	—	{ receipt, status: done } — stock +, ledger logged
/receipts/:id/cancel	POST	—	{ receipt, status: canceled }

Deliveries Endpoints

Endpoint	Method	Request Body (key fields)	Response / Notes
/deliveries	GET	status?, contact?, date?	[{id, reference, from_loc, to, contact, status}]

/deliveries	POST	delivery_address, lines:[{product_id, qty}], schedule_date	{ delivery, WH/OUT/XXXX, status: draft }
/deliveries/:id/validate	POST	—	{ delivery, status: done } or 409 if stock insufficient
/deliveries/:id/cancel	POST	—	{ delivery, status: canceled }

Other Core Endpoints

Endpoint	Method	Request Body (key fields)	Response / Notes
/transfers	POST	from_location_id, to_location_id, lines	{ transfer, WH/INT/XXXX }
/transfers/:id/validate	POST	—	{ transfer, status: done } — moves stock between locations
/adjustments	POST	product_id, location_id, counted_qty	{ adjustment, diff applied, WH/ADJ/XXXX }
/stock-ledger	GET	type?, ref?, status?, date_from?, date_to?	[all movements paginated]
/dashboard/stats	GET	—	{ total_in_stock, low_stock, out_of_stock, pending_receipts, pending_deliveries, transfers }
/warehouses	GET/POST/PUT	name, short_code, address	{ warehouse }
/locations	GET/POST/PUT	name, short_code, warehouse_id	{ location }

7. AI / ML Tasks — Detailed

Tech Stack: Python (FastAPI microservice or integrated) + scikit-learn / rule-based logic + REST integration with backend

Important: AI/ML ko over-engineer mat karo hackathon me. Rule-based + simple ML enough hai for 8 hours. Focus on integration.

AI Features Overview

AI Feature	Approach / Library	Input	Output
Low Stock Alert	Rule-based threshold check	current stock, reorder_level per product	Alert flag + list of products needing reorder
Out-of-Stock Detection on Delivery	Real-time stock check via DB query	product_id, requested_qty	Boolean flag + alert message for FE to show red line
Smart Reorder Suggestion	Simple moving average of past receipts (last 30 days)	stock_ledger history per product	Suggested reorder quantity
Demand Trend (Optional)	Linear regression on weekly outgoing stock	delivery history per product	Next week estimated demand
Reference Auto-Generation	DB MAX query + increment + format	operation_type, warehouse_short_code	WH/IN/0001 format string

AI-01 Low Stock Alert Engine [AI/ML]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	**60 min**	Dashboard — red alert badge on KPI card, Delivery — red product line highlight

What to build:

- Endpoint: GET /ai/low-stock-alerts
- Logic: Query products where stock_on_hand.quantity < products.reorder_level
- Return: list of { product_id, name, sku, current_stock, reorder_level, alert_severity: 'low' | 'out' }
- Alert severity: out = quantity <= 0, low = quantity < reorder_level
- Backend calls this after every validate (receipt, delivery, adjustment) to refresh alerts
- Can be scheduled: run check every 5 minutes and cache results in Redis (optional)
- Frontend polls /ai/low-stock-alerts on dashboard load and shows banner + red KPI card

Checklist / Deliverables:

- ☐ Endpoint returns correct products below threshold
- ☐ Severity differentiated (low vs out)
- ☐ Dashboard shows count of low stock items
- ☐ Integration with backend validate flow

AI-02 Out-of-Stock Check on Delivery Line [AI/ML]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	**30 min**	Delivery Detail Form — red line highlight when product not in stock

What to build:

- Endpoint: POST /ai/check-stock { product_id, location_id, requested_qty }
- Logic: Query stock_on_hand for this product+location. Compare free_to_use vs requested_qty
- Return: { available: true/false, on_hand: X, free_to_use: Y, deficit: Z }

- Backend already does this check — AI module can be the same logic exposed separately for FE to call on-the-fly when user adds a product to delivery form
- FE calls this endpoint when user enters quantity in delivery product line
- FE shows red background on line if available: false
- Delivery status auto-set to 'waiting' if any line has stock issue

Checklist / Deliverables:

- ☐ Real-time stock check on quantity entry
- ☐ Returns correct available vs requested
- ☐ FE highlights correctly

AI-03 Smart Reorder Quantity Suggestion [AI/ML]

Priority	Time Estimate	Screen / Module (from Mockup)
P2-HIGH	60 min	Products Page — suggestion badge next to reorder_level, or alert panel

What to build:

- Endpoint: GET /ai/reorder-suggestions
- Logic: For each product below reorder_level, look at last 30 days of stock_ledger (outgoing) to calculate average daily consumption
- Suggested reorder qty = (avg_daily_consumption * 14) — stock a 2-week buffer
- If insufficient history (< 5 records): use reorder_level * 2 as default suggestion
- Return: [{ product_id, name, current_stock, suggested_reorder_qty, reason }]
- Display as a suggestion card on Dashboard or Products page

Checklist / Deliverables:

- ☐ Suggestions calculated from actual ledger history
- ☐ Fallback for new products with no history
- ☐ Shown clearly on UI with reason

AI-04 Reference Number Auto-Generator [AI/ML]

Priority	Time Estimate	Screen / Module (from Mockup)
P1-MUST	20 min	All operation forms — Reference field auto-populated

What to build:

- Function: generate_reference(operation_type, warehouse_code)
- DB query: SELECT MAX(sequence_num) FROM stock_ledger WHERE operation_type = X
- Increment sequence, pad to 4 digits: 0001, 0002...
- Format: {warehouse_short_code}/{operation_code}/{sequence} — e.g. WH/IN/0001
- operation_code mapping: receipt=IN, delivery=OUT, transfer=INT, adjustment=ADJ
- Thread-safe: use DB sequence or transaction lock to avoid duplicates in concurrent requests
- Called automatically on POST /receipts, /deliveries, /transfers, /adjustments

Checklist / Deliverables:

- ☐ References generate in correct format
- ☐ No duplicates even under concurrent load
- ☐ Resets per operation type (IN has own sequence, OUT has own)

AI-05 Demand Trend Analysis (Optional/Stretch) [AI/ML]

Priority	Time Estimate	Screen / Module (from Mockup)
P3-NICE	45 min	Dashboard — small trend chart or insight card

What to build:

- Endpoint: GET /ai/demand-trends?product_id=X
- Logic: Group stock_ledger deliveries by week for last 8 weeks per product
- Simple linear regression (sklearn LinearRegression) to predict next week demand
- Return: { product_id, weekly_data: [...], predicted_next_week: X, trend: 'up'|'down'|'stable' }
- Show as a small insight card or tooltip on Products page
- Only implement if AI-01, AI-02, AI-03 are done and tested

Checklist / Deliverables:

- ☐ Weekly breakdown calculated correctly
- ☐ Prediction shown with trend direction
- ☐ Graceful fallback if insufficient data

8. Business Rules — Mockup Se Direct

Rule	Implementation Detail
Responsible = Auto-fill	Mockup says auto fill with current logged-in user. On any new receipt/delivery form, pre-populate Responsible field with JWT token's user name
Status: Receipts	Draft (initial) → click TODO → Ready → click Validate → Done. Mockup: 'To Do = When in Draft, Validate = When in Ready'
Status: Deliveries	Draft → Waiting (if out of stock) → Ready → Validate → Done. Mockup: 'Draft > Waiting > Ready > Done'
Late Operations	Mockup shows '1 Late, 6 operations' on dashboard. Late = schedule_date < today AND status not done
Waiting Operations	Delivery waiting for out-of-stock product to arrive. Resolved when receipt validates that product
Stock On Hand vs Free to Use	On Hand = total physical qty. Free to Use = On Hand minus qty reserved in pending delivery orders
Print Receipt	Mockup: 'Print the receipt once it's DONE'. Print button only active when status = done
Kanban View	Mockup explicitly says: allow user to switch to kanban view based on status. Receipts kanban: Draft / Ready / Done columns. Delivery: Draft / Waiting / Ready / Done
Move History Grouping	Mockup: 'if single reference has multiple products display it as...' — group rows by reference, show product-wise sub-rows
Low Stock Badge	Dashboard KPI card for Low/Out of Stock should show count. Products page should show alert badge. Delivery product line should go red
Reorder Rules	Each product has reorder_level. When stock falls below — trigger AI alert. Can set from product form
Multi-Warehouse	Receipts go to a specific warehouse location. Deliveries come from a specific location. All tracked per location in stock_on_hand table

9. Quick Reference Cheatsheet

FRONTEND	BACKEND	AI/ML
FE-01: Setup + Sidebar (45m) FE-02: Login + Signup (45m) FE-03: Dashboard (60m) FE-04: Products (60m) FE-05: Receipts (75m) FE-06: Deliveries (75m) FE-07: Move History (45m) FE-08: Settings (45m) FE-09: Adjustments (40m) **Total: ~7.5 hours**	BE-01: Auth + DB Schema (60m) BE-02: Products API (45m) BE-03: Receipts API (60m) BE-04: Deliveries API (60m) BE-05: Transfers + Ledger + Dashboard (60m) BE-06: Settings API (30m) **Total: ~5.5 hours (parallel)**	AI-01: Low Stock Alert (60m) AI-02: Out-of-Stock Check (30m) AI-03: Reorder Suggestion (60m) AI-04: Reference Generator (20m) AI-05: Demand Trend (45m, stretch) **Total: ~3.5 hours (parallel)**